# Review for Quiz 3

## Welcome back to CS 2100!

Prof. Rasika Bhalerao

## Topics revisited from previous quizzes

- Unit testing
  - Using `assert` and reading its test output
  - Using `pytest.raises()` and reading its test output
  - Identifying test cases
- Classes
  - Constructors, methods, attributes
  - `__str__()` and `__eq__()`

State and aliasing

## New topics on Quiz 3

- Lists, sets, and dictionaries
  - List comprehension
  - Iterating through lists, sets, and dictionaries
  - Rules about contents of lists, sets, and dictionaries
  - Sorting and filtering
  - Binary operators (`|`, `-`, etc.)
- Mean and variance
  - Formulas for mean, variance, and standard deviation

# List comprehension

Move the body of a `for loop` to right before the `for` (after the opening bracket `[`)

```
my_nums: List[int] = [6, 7, 8, 9]

increased_nums: List[int] = [i + 1 for i in my_nums] # list comprehension

print(increased_nums)  # [7, 8, 9, 10]
```

If we want the resulting list to **filter elements**, we add the `if` clause after the `for` clause.

```
def positive_copy(nums: List[int]) -> List[int]:
    return [i for i in nums if i >= 0]
```

## Poll: Which list comprehension correctly creates a list of only the even numbers from 1 to 10?

1. `[x if x % 2 == 0 for x in range(1, 11)]`
2. `[x for x in range(1, 11) where x % 2 == 0]`
3. `[x for x in range(1, 11) if x % 2 == 0]`
4. `[x % 2 == 0 for x in range(1, 11)]`

# Sorting

```
words: List[str] = 'never gonna give you up'.split()

sorted_alphabetically: List[str] = sorted(words)
print(' '.join(sorted_alphabetically)) # give gonna never up you

sorted_by_length: List[str] = sorted(words, key = lambda word: len(word))
print(' '.join(sorted_by_length))  # up you give gonna never
```

The `sorted()` function has an optional argument `key`, which is a function that is applied to each element when determining the sorted order.

## Poll: Given this list of tuples representing students and their scores:

```
[('Mini', 85), ('Binnie', 92), ('Ginnie', 78), ('Spleenie', 92)]
```

## Which code will sort the students by score in descending order (highest to lowest)?

1. `sorted(students, key=lambda x: -x[1], reverse=True)`
2. `sorted(students, key=lambda x: x[1], reverse=True)`
3. `sorted(students, key=lambda x: -x[1])`
4. `sorted(students, reverse=True)`

# Dictionaries

- Maps `key` --> `value`
- Can have the same `value` twice, but not the same `key` twice ( `key` s are a set)
- If we map a `key` to a `value` , and then later on, map the same `key` to a different `value` , it overwrites the old `value` with the new one
- `dictionary['cat']` vs `dictionary.get('cat', 4)` :
  - get method will return `4` if `'cat'` isn't in the dictionary (or `None` if we didn't specify the `4` )
  - brackets will raise `KeyError` if `'cat'` isn't in the dictionary

## Poll: What happens?

```
student_grades = {"Binnie": 85, "Ginnie": 92, "Mini": 78}

result1 = student_grades.get("Spleenie")
result2 = student_grades.get("Spleenie", 0)
result3 = student_grades["Spleenie"]
```

1. All three results will be None
2. `result1` is `None`, `result2` is 0, and the third result line raises a `KeyError`
3. The first result line raises a `KeyError`, `result2` is 0, and `result3` is `None`
4. All three result lines raise a `KeyError` because 'Spleenie' is not in the dictionary

# Dictionary syntax: iteration

```
ages: Dict[str, int] = {'cat': 10, 'elephant': 14, 'dog': 3}
```

- iterate over its `key` s

```
for key in ages:
    print(f"{key}'s age is {ages.get(key)}")
```

- iterate over its `key-value` pairs

```
for key, value in ages.items():
    print(f"{key}'s age is {value}")
```

# Poll: Which correctly prints each student's name and grade?

```
grades = {"Binnie": 85, "Ginnie": 92, "Mini": 78}
```

1.

```
for student in grades:
    print(student, grades[student])
```

2.

```
for student, grade in grades.items():
    print(student, grade)
```

3.

```
for pair in grades:
    print(pair[0], pair[1])
```

4.

```
for pair in grades:
    print(pair[0], pair[1])
```

# Set operations

- Union (`a | b`): a set that has all elements that are in either set `a` or set `b`
- Intersection (`a & b`): a set that has all elements that are in both set `a` and set `b`
- Subset (`a <= b`): `True` if all elements in `a` are also in `b`, and `False` otherwise
  - Strict subset (`a < b`): `True` if `a <= b` **and `a` is not equal to `b`**, and `False` otherwise
- Subtraction (`a - b`): a set that has all elements in `a` that are not in `b`

## Poll: What is in `result1` and `result2`?

```
students_in_math = {'Mini', 'Binnie', 'Ginnie'}
students_in_cs = {'Mini', 'Mega', 'Micro'}

result1 = students_in_math & students_in_cs
result2 = students_in_math | students_in_cs
```

1.

```
result1 = {'Mini'}
result2 = {'Mini', 'Binnie',
    'Ginnie', 'Mega', 'Micro'}
```

2.

```
result1 = {'Mini', 'Binnie',
    'Ginnie', 'Mega', 'Micro'}
result2 = {'Mini'}
```

3.

```
result1 = {'Mini'}
result2 = {'Mini'}
```

4.

```
result1 = {'Mini', 'Binnie',
    'Ginnie', 'Mega', 'Micro'}
result2 = {'Mini', 'Binnie',
    'Ginnie', 'Mega', 'Micro'}
```

# Mean, variance, and standard deviation

Mean = sum of all values ÷ number of values

Variance = mean of the squared deviations from the mean

Standard deviation = square root of variance

**Poll: Two sorting algorithms both have a mean runtime of 50ms. Algorithm A has a standard deviation of 2ms. Algorithm B has a standard deviation of 40ms. Which algorithm has a more consistent runtime?**

1. Algorithm A
2. Algorithm B
3. They're equally consistent
4. Cannot be determined from this information

**Poll: Lists A and B each have a mean of 40. List A has 5 numbers in it, and List B has 8 numbers. If we add the number 50 to both lists (as a sixth and ninth number, respectively), which will have a higher mean?**

1. List A
2. List B
3. They will have the same mean after this
4. Cannot be determined

**Poll: Lists A and B each have a standard deviation of 30. List A has 5 numbers in it, and List B has 8 numbers. If we add the number 50 to both lists (as a sixth and ninth number, respectively), which will have a higher standard deviation?**

1. List A
2. List B
3. They will have the same standard deviation after this
4. Cannot be determined

# Practice Quiz 3 questions... but more

```python
class Triangle:
    """A triangle class that represents a triangle with three sides."""
    def __init__(self, a: float = 1, b: float = 1, c: float = 1) -> None:
        """Initialize the triangle with three sides.
        Args:
            a (float): The length of side a.
            b (float): The length of side b.
            c (float): The length of side c.
        """
        self.a = a
        self.b = b
        self.c = c
```

## Poll: which is incorrect?

1.

```python
def is_equilateral(self) -> bool:
    return self.a == self.b == self.c
```

2.

```python
def is_equilateral(self) -> bool:
    return len({self.a, self.b, self.c}) == 1
```

3.

```python
def is_equilateral(self) -> bool:
    return len({a, b, c}) == 1
```

4.

```python
def is_equilateral(self) -> bool:
    a, b, c = self.a, self.b, self.c
    return a == b == c
```

```python
class Triangle:
    """A triangle class that represents a triangle with three sides."""
    def __init__(self, a: float = 1, b: float = 1, c: float = 1) -> None:
        """Initialize the triangle with three sides.
        Args:
            a (float): The length of side a.
            b (float): The length of side b.
            c (float): The length of side c.
        """
        self.a = a
        self.b = b
        self.c = c
```

## Poll: which is incorrect?

1.

```python
def is_isosceles(self) -> bool:
    return self.a == self.b or self.a == self.c or self.b == self.c
```

2.

```python
def is_isosceles(self) -> bool:
    a, b, c = self.a, self.b, self.c
    return a == b or a == c or b == c
```

3.

```python
def is_isosceles(self) -> bool:
    return len({self.a, self.b, self.c}) != 2 and not is_equilateral()
```

4.

```python
def is_isosceles(self) -> bool:
    return len({self.a, self.b, self.c}) != 2 and not self.is_equilateral()
```

# Poll: Which unit test cases can we come up with for this constructor?

```python
class Triangle:
    """A triangle class that represents a triangle with three sides."""

    def __init__(self, a: float = 1, b: float = 1, c: float = 1) -> None:
        """Initialize the triangle with three sides.
        Args:
            a (float): The length of side a.
            b (float): The length of side b.
            c (float): The length of side c.

        Raises:
            ValueError: If the sides do not form a valid triangle.
        """
        if (a + b <= c) or (a + c <= b) or (b + c <= a) or (a <= 0) or (b <= 0) or (c <= 0):
            raise ValueError(f"Invalid triangle sides: {a}, {b}, {c}")
        self.a = a
        self.b = b
        self.c = c
```

```
class Square:
    """A class representing a square."""

    def __init__(self, side: float) -> None:
        """Initialize a square with the given side length.
        Args:
        side : float
        The length of the side of the square.
        """
        self.side = side

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Square):
            return False
        return self.side == other.side

square1 = Square(1.0)
```

## Poll: Which of these is equal to square1?

1. `square2 = square1`
2. `square3 = Square(1.0)`
3. `square4 = Square(square1.side)`
4. `square5 = Square(500.0)`

Poll: A list of 11 numbers has a mean of 30 and a variance of 20. If we remove one of the values, which equals 30, what are the new mean and variance?

## Poll: How did we calculate the new mean in the previous question?

1. We determined what all of the original numbers are and removed the extra number to calculate the new mean
2. We used the variance to calculate how much the mean would vary
3. We determined the sum of the previous numbers, subtracted the extra number, and divided by the new number of numbers
4. We didn't calculate the new mean

Consider the scenario where you are across town, and you need to take the bus or bike to a class that starts in 20 minutes. Suppose that biking there takes on average 18 minutes, and taking the bus takes on average 14 minutes.

A student says, "Taking the bus has a mean 4 minutes shorter, so it must be the better choice for arriving on time." Give a scenario where the bus is the right choice, and one where the bus is the wrong choice.

### Poll: Which of these would not satisfy the requirements for the "wrong choice" part?

1. taking the bus has a large standard deviation (e.g., 10 minutes), and biking has a small standard deviation (e.g., 1 minute)
2. taking the bus has a small standard deviation (e.g., 1 minute), and biking has a large standard deviation (e.g., 10 minutes)
3. taking the bus and biking have the same standard deviation (e.g., 1 minute)

## Not a poll, but an exercise:

Write a function called invert that takes a dictionary (where the keys and values are both words) and returns a new dictionary where each value of the original becomes a key in the new dictionary, and the value associated with that key is the list of all original keys that mapped to it. Documentation is not graded for this question.

```
def invert(d: dict[str, str]) -> dict[str, list[str]]:
    """Invert a dictionary mapping strings to strings, producing a dictionary mapping strings to
    lists of strings.

    Args:
        d (dict[str, str]): The dictionary to invert.
    Returns:
        dict[str, list[str]]: The inverted dictionary.
    """
    inverted: dict[str, list[str]] = {}
    for key, value in d.items():
        if value not in inverted:
            inverted[value] = []
        inverted[value].append(key)
    return inverted
```

## Poll:

1. What is your main takeaway from today?

2. What would you like to revisit next time?